

Laporan Praktikum Pemrograman Berorientasi Objek

Pekan 3

Lanjutan Membuat Fungsi CRUD (Create, Read, Update, Delete) User Dengan Database MySql



Oleh : Iqbal Miftahul Fikri

NIM 2411531004

Dosen Pengampu : Nurfiah, S.ST, M.Kom.

Fakultas Teknologi Informasi

Departemen Informatika

Universitas Andalas

Padang

2025

A. Pendahuluan

Di berbagai sektor industri saat ini, pengembangan perangkat lunak memegang peranan penting. Bahasa pemrograman Java populer dan dikenal fleksibel sehingga sering dipakai untuk membangun aplikasi dari yang sederhana hingga sistem berskala besar. Tantangan utama dalam pengembangan adalah menjaga agar kode tetap teratur, mudah diperbaiki, dan memungkinkan pengembangan lanjutan tanpa kesulitan. Jika desainnya kurang baik, aplikasi bisa menjadi rumit dan sulit diubah tanpa menimbulkan masalah baru.

Untuk mengatasi hal tersebut, pengembang memakai pola arsitektur dan desain tertentu yang memberikan solusi efektif terhadap masalah umum dalam pembuatan perangkat lunak. Salah satu prinsip utama adalah pemisahan tanggung jawab (Separation of Concerns), yang diterapkan melalui pola seperti Model-View-Controller (MVC) dan Data Access Object (DAO). Dengan memisahkan bagian data, logika bisnis, dan tampilan antarmuka, aplikasi menjadi lebih terstruktur, mudah dikelola, dan siap untuk dikembangkan lebih lanjut.

B. Tujuan

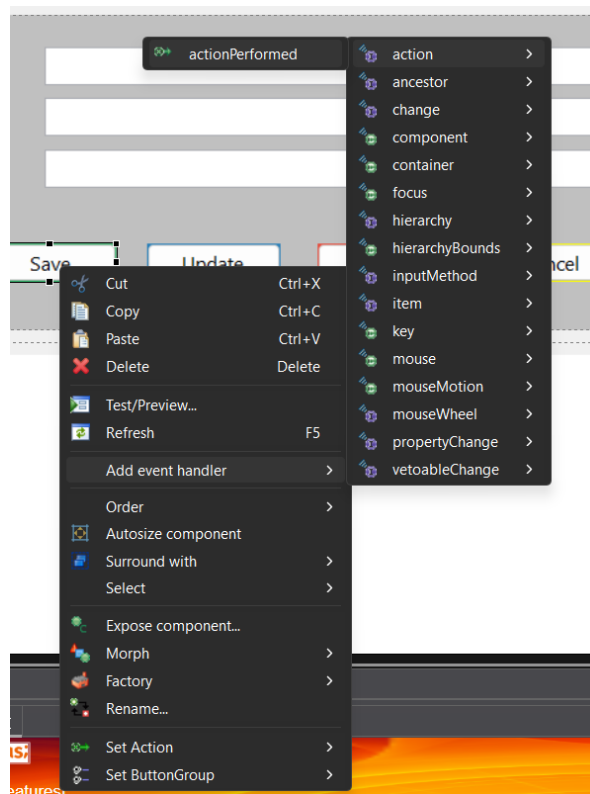
Tujuan dari praktikum ini adalah :

1. Menyediakan kemudahan dalam pengelolaan data pelanggan secara terorganisir melalui operasi CRUD (Create, Read, Update, Delete).
2. Menghadirkan antarmuka aplikasi yang simpel dan friendly agar pengguna dapat dengan mudah menambah, melihat, mengubah, dan menghapus data pelanggan secara efisien.
3. Meningkatkan efektifitas pencatatan dan pemeliharaan data pelanggan sehingga tidak perlu dilakukan secara manual.

C. Langkah – Langkah

a. Create User

1. Pertama-tama klik kanan pada tombol save yang ada pada design UserFrame
→ add event handlers → actionPerformed.



2. Setelah itu, buat kode program seperti gambar dibawah ini.

```
User user = new User();
user.setNama(txtName.getText());
user.setUsername(txtUsername.getText());
user.setPassword(txtPassword.getText());
usr.save(user);
reset();
```

Kode ini mengambil data dari kolom isian form dan menyimpannya ke database. Pertama, dibuat objek User baru yang kosong. Lalu, data yang diinput di txtName, txtUsername, dan txtPassword diambil dan dimasukkan ke objek User tersebut. Setelah penuh, objek User dikirim ke metode save milik objek usr untuk disimpan. Terakhir, fungsi reset() dipanggil untuk mengosongkan form agar siap digunakan untuk input data berikutnya.

b. Read Users

1. Buat sebuah method dengan nama loadTable(), lalu isi method tersebut dengan kode program berikut.

```
public void loadTable() {
    ls = usr.show();
    TableUser tu = new TableUser(ls);
    tableUsers.setModel(tu);
    tableUsers.getTableHeader().setVisible(true);
}
```

Kode ini memuat data pengguna dari sumber database dan menampilkannya di tabel tableUsers. Pertama, kode memanggil metode `usr.show()` untuk mengambil semua data pengguna dan menyimpannya dalam list `ls`. List ini kemudian dipakai untuk membuat model tabel `tu` dari kelas `TableUser`, yang mengatur tampilan data dalam tabel. Model `tu` selanjutnya dipasang ke `tableUsers` dengan `.setModel()`, sehingga data pengguna terlihat secara lengkap. Terakhir, kode memastikan header kolom pada tabel tampil dengan benar.

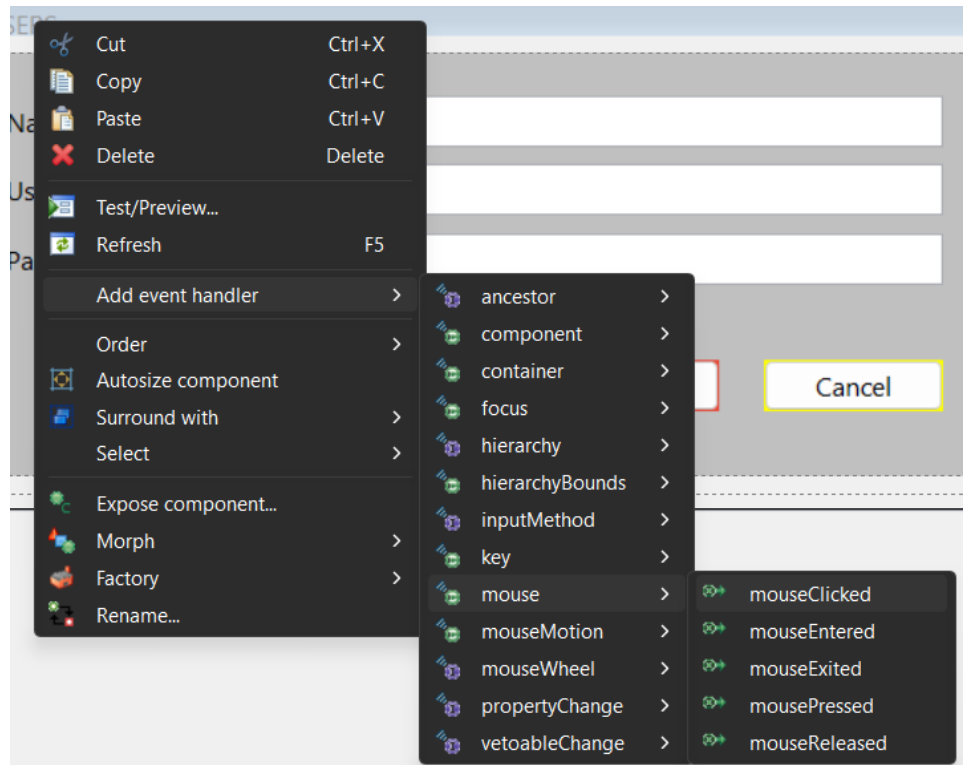
2. Memanggil metode dari class main, sehingga saat program pertama kali dijalankan, fungsi `loadTable` akan otomatis dijalankan.

```
UserFrame frame = new UserFrame();
frame.setVisible(true);
frame.loadTable();
```

Kode ini menggambarkan cara kerja aplikasi manajemen pengguna sederhana. Saat aplikasi dijalankan, jendela utama (`UserFrame`) dibuat dan langsung ditampilkan. Metode `loadTable` otomatis dipanggil untuk mengambil data pengguna dari database dan menampilkannya di tabel. Pengguna bisa memilih baris tabel untuk melihat atau mengubah data di form isian. Setelah data diperbarui, pengguna dapat menyimpannya. Data dari form akan dikumpulkan ke dalam objek `User`, disimpan ke database, lalu form dikosongkan agar siap untuk input data berikutnya.

c. Update User

1. Klik kanan pada `JTable` yang ada pada `UserFrame` → add event handler → mouse → mouse Clicked.



```

165 public void mouseClicked(MouseEvent e) {
166     id = tableUsers.getValueAt(tableUsers.getSelectedRow(), 0).toString();
167     txtName.setText(tableUsers.getValueAt(tableUsers.getSelectedRow(), 1).toString());
168     txtUsername.setText(tableUsers.getValueAt(tableUsers.getSelectedRow(), 2).toString());
169     txtPassword.setText(tableUsers.getValueAt(tableUsers.getSelectedRow(), 3).toString());
170 }

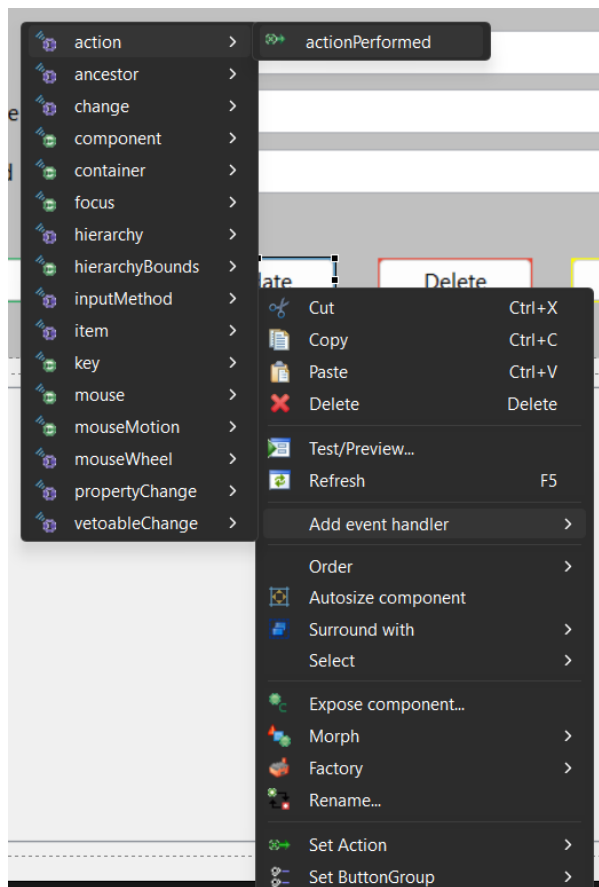
```

Kode ini mengambil data dari baris yang dipilih di tabel dan menampilkannya ke dalam form isian. Saat pengguna mengklik baris pada tabel tableUsers, kode akan mengambil nilai dari setiap kolom di baris tersebut: nilai kolom pertama disimpan ke variabel id, lalu nilai kolom kedua, ketiga, dan keempat dimasukkan ke field txtName, txtUsername, dan txtPassword. Fungsi ini sering dipakai agar pengguna bisa dengan mudah melihat atau mengubah data dari daftar yang dipilih.

2. Saat salah satu baris di tabel diklik, data tersebut akan langsung muncul di form input.

5	Iqbal Miftahul Fikri	BallFord	12345
6	Jiyad Pasanibu	Jiyad	54321

- Setelah itu, klik kanan tombol update pada design UserFrame → add event handler → action → actionPerformed.



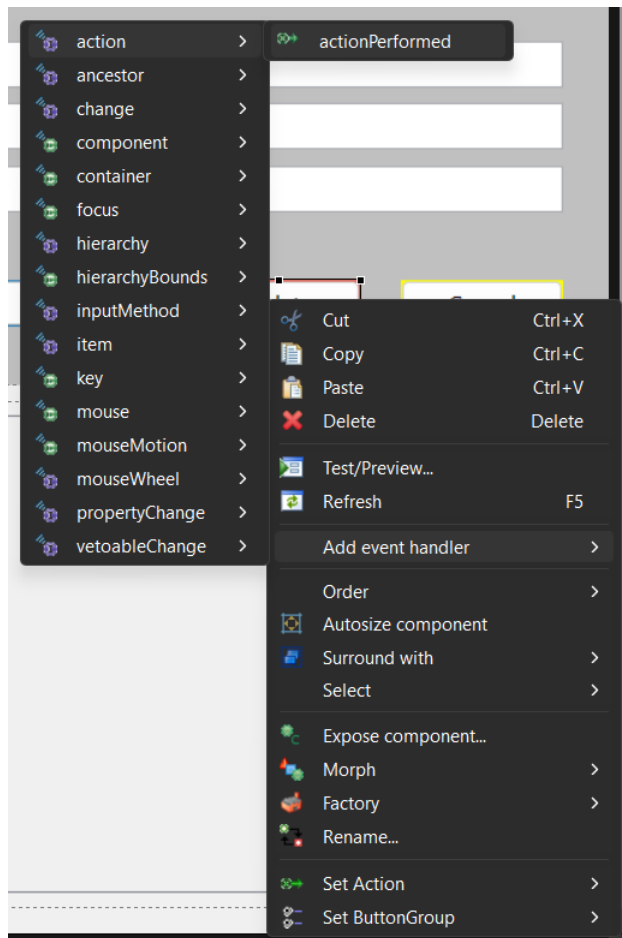
4. Lalu isikan kode program seperti gambar berikut.

```
120 public void actionPerformed(ActionEvent e) {  
121     User user = new User();  
122     user.setNama(txtName.getText());  
123     user.setUsername(txtUsername.getText());  
124     user.setPassword(txtPassword.getText());  
125     user.setId(id);  
126     usr.update(user);  
127     reset();  
128     loadTable();  
129 }
```

Kode ini berfungsi untuk memperbarui data pengguna yang sudah ada. Data yang diubah dari form txtName, txtUsername, dan txtPassword dimasukkan ke objek User baru, lalu ID pengguna yang ingin diperbarui disetel dengan `user.setId(id)`. Setelah itu, metode `usr.update()` dipanggil untuk menyimpan perubahan ke database. Terakhir, form dikosongkan dengan `reset()` dan tabel diperbarui dengan `loadTable()` agar data terbaru langsung terlihat.

d. Delete User

1. Klik kanan tombol delete pada design UserFrame → add event handler → action → actionPerformed.



2. Lalu isikan kode seperti gambar berikut.

```

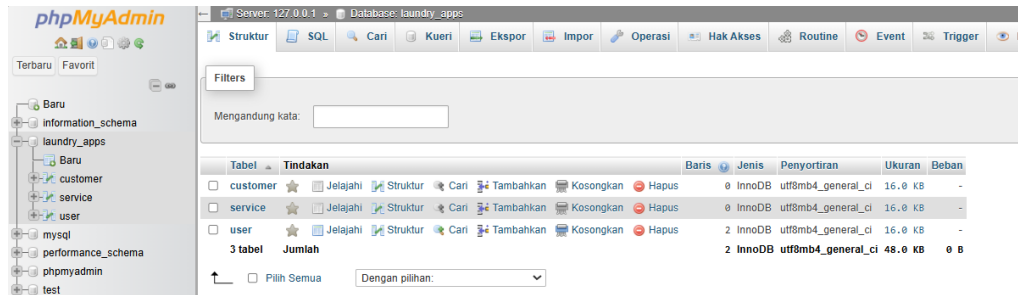
151     if(id != null) {
152         usr.delete(id);
153         reset();
154         loadTable();
155     } else {
156         JOptionPane.showMessageDialog(null, "Silahkan pilih data yang akan di hapus");
157     }

```

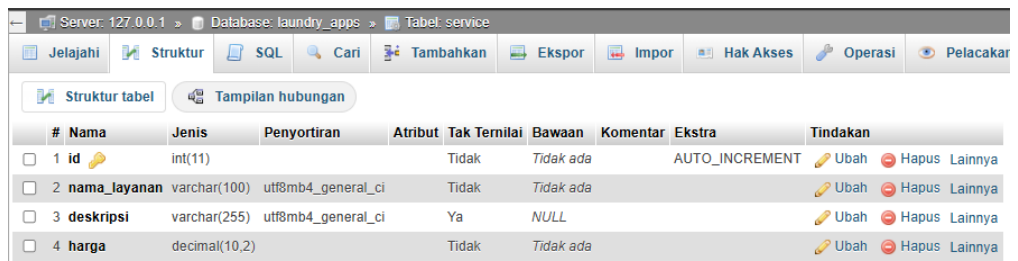
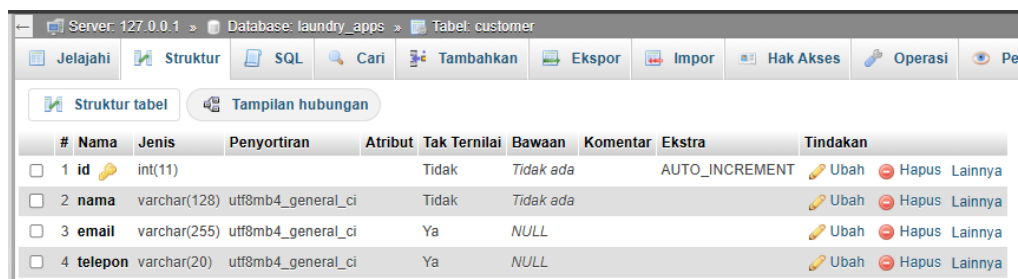
Kode ini digunakan untuk menghapus data pengguna dengan pengecekan penting. Pertama, kode memeriksa apakah pengguna sudah memilih baris di tabel dengan melihat apakah variabel id berisi nilai. Jika id ada, kode memanggil metode `usr.delete(id)` untuk menghapus data dari database, kemudian mengosongkan form dengan `reset()` dan memuat ulang tabel dengan `loadTable()` agar hasil penghapusan terlihat. Jika belum ada pilihan data (id kosong), program menampilkan pesan "Silahkan pilih data yang akan di hapus".

e. Latihan / Tugas

1. Pertama-tama bikin tabel di database pada phpMyadmin dengan nama customer dan service.



Begini struktur masing-masing nya.



2. Buat class baru pada package DAO dengan nama CustomerRepo, isi dengan kodingan seperti ini.

```

1 package DAO;
2
3 import java.sql.Connection;
4 import java.sql.DriverManager;
5 import java.sql.PreparedStatement;
6 import java.sql.ResultSet;
7 import java.sql.SQLException;
8 import java.sql.Statement;
9 import java.util.ArrayList;
10 import java.util.List;
11 import model.Customer;
12
13 public class CustomerRepo {
14
15     private static final String DB_URL = "jdbc:mysql://localhost:3306/laundry_apps";
16     private static final String DB_USER = "root";
17     private static final String DB_PASSWORD = "";
18
19     private Connection getConnection() throws SQLException {
20         return DriverManager.getConnection(DB_URL, DB_USER, DB_PASSWORD);
21     }
22
23     public List<Customer> show() {
24         List<Customer> customerList = new ArrayList<>();
25         String sql = "SELECT * FROM customer";
26
27         try (Connection conn = getConnection();
28             Statement stmt = conn.createStatement();
29             ResultSet rs = stmt.executeQuery(sql)) {
30
31             while (rs.next()) {
32                 Customer customer = new Customer();
33                 customer.setId(String.valueOf(rs.getInt("id")));
34                 customer.setNama(rs.getString("nama"));
35                 customer.setEmail(rs.getString("email"));
36                 customer.setTelepon(rs.getString("telepon"));
37                 customerList.add(customer);
38             }
39         } catch (SQLException e) {
40             System.err.println("Gagal mengambil data pelanggan: " + e.getMessage());
41             e.printStackTrace();
42         }
43         return customerList;
44     }
45
46     public void save(Customer customer) {
47         String sql = "INSERT INTO customer (nama, email, telepon) VALUES (?, ?, ?)";
48
49         try (Connection conn = getConnection();
50             PreparedStatement pstmt = conn.prepareStatement(sql)) {
51
52             pstmt.setString(1, customer.getNama());
53             pstmt.setString(2, customer.getEmail());
54             pstmt.setString(3, customer.getTelepon());
55             pstmt.executeUpdate();
56             System.out.println("Data berhasil disimpan ke database!");
57         } catch (SQLException e) {
58             System.err.println("Gagal menyimpan data: " + e.getMessage());
59             e.printStackTrace();
60         }

```

```

61     }
62 }
63
64 public void update(Customer customer) {
65     String sql = "UPDATE customer SET nama = ?, email = ?, telepon = ? WHERE id = ?";
66
67     try (Connection conn = getConnection();
68         PreparedStatement pstmt = conn.prepareStatement(sql)) {
69
70         pstmt.setString(1, customer.getNama());
71         pstmt.setString(2, customer.getEmail());
72         pstmt.setString(3, customer.getTelepon());
73         pstmt.setInt(4, Integer.parseInt(customer.getId()));
74         pstmt.executeUpdate();
75         System.out.println("Data berhasil diupdate!");
76
77     } catch (SQLException e) {
78         System.err.println("Gagal mengupdate data: " + e.getMessage());
79         e.printStackTrace();
80     }
81 }
82
83 public void delete(String id) {
84     String sql = "DELETE FROM customer WHERE id = ?";
85
86     try (Connection conn = getConnection();
87         PreparedStatement pstmt = conn.prepareStatement(sql)) {
88
89         pstmt.setInt(1, Integer.parseInt(id));
90         pstmt.executeUpdate();
91         System.out.println("Data berhasil dihapus!");
92
93     } catch (SQLException e) {
94         System.err.println("Gagal menghapus data: " + e.getMessage());
95         e.printStackTrace();
96     }
97 }
98 }

```

Kode ini berperan sebagai pengelola data yang menghubungkan aplikasi dengan tabel customer di database laundry_apps. Setiap metode di dalamnya punya fungsi khusus: show() untuk menampilkan semua data pelanggan, save() untuk menambahkan pelanggan baru, update() untuk memperbarui informasi pelanggan yang sudah ada, dan delete() untuk menghapus data pelanggan berdasarkan ID.

3. Buat class baru pada package DAO dengan nama ServiceRepo, isi kodingan seperti ini.

```

1 package DAO;
2
3 import java.sql.Connection;
4 import java.sql.PreparedStatement;
5 import java.sql.ResultSet;
6 import java.sql.SQLException;
7 import java.util.ArrayList;
8 import java.util.List;
9 import model.Service;
10 import config.Database;
11
12 public class ServiceRepo {
13
14     public void save(Service service) {
15
16         String sql = "INSERT INTO service (nama_layanan, deskripsi, harga) VALUES (?, ?, ?)";
17         try (Connection conn = Database.koneksi();
18             PreparedStatement pstmt = conn.prepareStatement(sql)) {
19             pstmt.setString(1, service.getServiceName());
20             pstmt.setString(2, service.getDescription());
21             pstmt.setDouble(3, service.getHarga());
22             pstmt.executeUpdate();
23         } catch (SQLException e) {
24             e.printStackTrace();
25         }
26     }
27
28     public List<Service> show() {
29         List<Service> services = new ArrayList<>();
30         String sql = "SELECT * FROM service";
31         try (Connection conn = Database.koneksi();
32             PreparedStatement pstmt = conn.prepareStatement(sql);
33             ResultSet rs = pstmt.executeQuery()) {
34             while (rs.next()) {
35                 Service service = new Service();
36                 service.setId(rs.getString("id"));
37                 service.setServiceName(rs.getString("nama_layanan"));
38                 service.setDescription(rs.getString("deskripsi"));
39                 service.setHarga(rs.getDouble("harga"));
40                 services.add(service);
41             }
42         } catch (SQLException e) {
43             e.printStackTrace();
44         }
45         return services;
46     }
47
48     public void update(Service service) {
49         String sql = "UPDATE service SET nama_layanan = ?, deskripsi = ?, harga = ? WHERE id = ?";
50         try (Connection conn = Database.koneksi();
51             PreparedStatement pstmt = conn.prepareStatement(sql)) {
52             pstmt.setString(1, service.getServiceName());
53             pstmt.setString(2, service.getDescription());
54             pstmt.setDouble(3, service.getHarga());
55             pstmt.setInt(4, Integer.parseInt(service.getId()));
56             pstmt.executeUpdate();
57         } catch (SQLException e) {
58             e.printStackTrace();
59         }
60     }
61
62     public void delete(String id) {
63         String sql = "DELETE FROM service WHERE id = ?";
64         try (Connection conn = Database.koneksi();
65             PreparedStatement pstmt = conn.prepareStatement(sql)) {
66             pstmt.setInt(1, Integer.parseInt(id));
67             pstmt.executeUpdate();
68         } catch (SQLException e) {
69             e.printStackTrace();
70         }
71     }
72 }

```

Kode ServiceRepo ini berfungsi sebagai pengelola data khusus untuk tabel service di database, mirip dengan CustomerRepo sebelumnya. Kode ini menangani semua operasi yang berkaitan dengan data layanan laundry, seperti menambahkan jenis layanan baru, menampilkan semua layanan yang tersedia, memperbarui harga atau deskripsi layanan, serta menghapus layanan dari daftar.

- Setelah itu pada package model, buat class baru dengan nama Customer, isi kodingan seperti ini.

```
1 package model;
2
3 public class Customer {
4     String id;
5     String nama;
6     String alamat;
7     String nomorHp;
8     String email;
9     String telepon;
10
11     public String getId() {
12         return id;
13     }
14
15     public void setId(String id) {
16         this.id = id;
17     }
18
19     public String getNama() {
20         return nama;
21     }
22
23     public void setNama(String nama) {
24         this.nama = nama;
25     }
26
27     public String getAlamat() {
28         return alamat;
29     }
30
31     public void setAlamat(String alamat) {
32         this.alamat = alamat;
33     }
34
35     public String getNomorHp() {
36         return nomorHp;
37     }
38
39     public void setNomorHp(String nomorHp) {
40         this.nomorHp = nomorHp;
41     }
42
43     public void setEmail(String email) {
44         this.email = email;
45     }
46
47     public void setTelepon(String telepon) {
48         this.telepon = telepon;
49     }
50
51     public String getEmail() {
52         return email;
53     }
54
55     public String getTelepon() {
56         return telepon;
57     }
58
59 }
60
```

Kode ini merupakan sebuah model atau cetakan (blueprint) untuk data pelanggan. Di dalamnya, terdapat definisi berbagai properti yang dimiliki pelanggan, seperti id, nama, alamat, email, dan lain-lain. Metode get (misalnya

getNama()) digunakan untuk mengambil data dari objek, sementara metode set (misalnya setName()) dipakai untuk mengisi atau mengubah data tersebut.

5. Buat class baru pada package model dengan nama Service, isi kodingan seperti ini.

```
1  package model;
2
3  public class Service {
4      String id;
5      String jenis;
6      double harga;
7      String status;
8      String serviceName;
9      String description;
10
11  public String getId() {
12      return id;
13  }
14
15  public void setId(String id) {
16      this.id = id;
17  }
18
19  public String getJenis() {
20      return jenis;
21  }
22
23  public void setJenis(String jenis) {
24      this.jenis = jenis;
25  }
26
27  public double getHarga() {
28      return harga;
29  }
30
31  public void setHarga(double harga) {
32      this.harga = harga;
33  }
34
35  public String getStatus() {
36      return status;
37  }
38
39  public void setStatus(String status) {
40      this.status = status;
41  }
42
43  public String getServiceName() {
44      return serviceName;
45  }
46
47  public String getDescription() {
48      return description;
49  }
50
51  public void setServiceName(String serviceName) {
52      this.serviceName = serviceName;
53  }
54
55  public void setDescription(String description) {
56      this.description = description;
57  }
58
59  }
60
```

Kode ini adalah sebuah model atau cetakan (blueprint) untuk data layanan, mirip seperti kelas Customer sebelumnya. Di dalamnya, didefinisikan berbagai informasi yang dimiliki layanan, seperti id, jenis layanan, harga, status, nama layanan dan deskripsi. Metode get digunakan untuk mengambil data dari objek layanan, sedangkan metode set berfungsi untuk mengisi atau mengubah data tersebut.

6. Selanjutnya, buat class baru dengan nama TableCustomer pada package table, lalu isi kodingan seperti ini.

```
1 package table;
2
3 import java.util.List;
4 import javax.swing.table.AbstractTableModel;
5 import model.Customer;
6
7 public class TableCustomer extends AbstractTableModel {
8
9     private List<Customer> list;
10
11     public TableCustomer(List<Customer> list) {
12         this.list = list;
13     }
14
15     @Override
16     public int getRowCount() {
17         return list.size();
18     }
19
20     @Override
21     public int getColumnCount() {
22         return 4;
23     }
24
25     @Override
26     public Object getValueAt(int rowIndex, int columnIndex) {
27         switch (columnIndex) {
28             case 0: return list.get(rowIndex).getId();
29             case 1: return list.get(rowIndex).getNama();
30             case 2: return list.get(rowIndex).getEmail();
31             case 3: return list.get(rowIndex).getTelepon();
32             default: return null;
33         }
34     }
35
36     @Override
37     public String getColumnName(int column) {
38         switch (column) {
39             case 0: return "ID";
40             case 1: return "Nama Pelanggan";
41             case 2: return "Email";
42             case 3: return "No. Telepon";
43             default: return null;
44         }
45     }
46 }
```

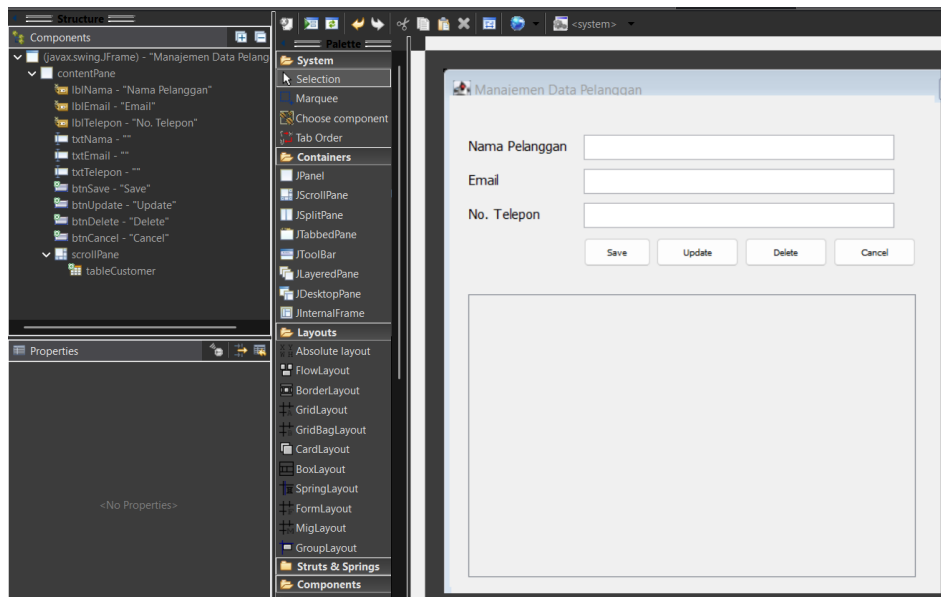
Kode ini berfungsi sebagai model tabel yang menghubungkan data mentah (List<Customer>) dengan komponen tabel visual (JTable) di antarmuka pengguna. Kode ini memberi tahu JTable tentang jumlah baris yang harus ditampilkan (getRowCount), jumlah kolom yang tersedia (getColumnCount), data apa yang harus ditampilkan di setiap sel (getValueAt), serta nama untuk tiap kolom (getColumnName).

7. Buat class baru dengan nama TableService pada package table, lalu isi kodingan seperti ini.

```
1 package table;
2
3 import java.util.List;
4 import javax.swing.table.AbstractTableModel;
5 import model.Service;
6
7 public class TableService extends AbstractTableModel {
8     private List<Service> services;
9     private final String[] columnNames = {"ID", "Service Name", "Description", "Harga"};
10
11     public TableService(List<Service> services) {
12         this.services = services;
13     }
14
15     @Override
16     public int getRowCount() {
17         return services.size();
18     }
19
20     @Override
21     public int getColumnCount() {
22         return columnNames.length;
23     }
24
25     @Override
26     public String getColumnName(int column) {
27         return columnNames[column];
28     }
29
30     @Override
31     public Object getValueAt(int rowIndex, int columnIndex) {
32         Service service = services.get(rowIndex);
33         switch (columnIndex) {
34             case 0:
35                 return service.getId();
36             case 1:
37                 return service.getServiceName();
38             case 2:
39                 return service.getDescription();
40             case 3:
41                 return service.getHarga();
42             default:
43                 return null;
44         }
45     }
46 }
```

Kode ini berfungsi sebagai model tabel yang menghubungkan data List<Service> dengan komponen JTable untuk menampilkan informasi layanan. Kode ini menginformasikan jumlah baris (getRowCount), jumlah dan nama kolom (getColumnCount dan getColumnName), serta data spesifik seperti ID, nama layanan, deskripsi, atau harga yang harus ditampilkan di setiap sel (getValueAt). Kode ini berperan sebagai penata tata letak yang mengatur bagaimana daftar layanan ditampilkan dalam tabel.

8. Pada package ui, buat class JFrame baru dengan nama CustomerFrame. Lalu design dan isi kodingan nya seperti ini.



```

1  package ui;
2
3  import java.awt.EventQueue;
4  import javax.swing.JFrame;
5  import javax.swing.JPanel;
6  import javax.swing.border.EmptyBorder;
7  import DAO.CustomerRepo;
8  import table.TableCustomer;
9  import model.Customer;
10 import javax.swing.JLabel;
11 import java.awt.Font;
12 import java.util.List;
13 import javax.swing.JTextField;
14 import javax.swing.JScrollPane;
15 import javax.swing.JButton;
16 import javax.swing.JTable;
17 import java.awt.event.MouseAdapter;
18 import java.awt.event.MouseEvent;
19 import javax.swing.JOptionPane;
20
21 public class CustomerFrame extends JFrame {
22
23     private static final long serialVersionUID = 1L;
24     private JPanel contentPane;
25     private JTextField txtNama;
26     private JTextField txtEmail;
27     private JTextField txtTelepon;
28     private JTable tableCustomer;
29     private CustomerRepo customerRepo = new CustomerRepo();
30     private List<Customer> ls;
31     private String selectedId;
32
33     public static void main(String[] args) {
34         EventQueue.invokeLater(() -> {
35             try {
36                 CustomerFrame frame = new CustomerFrame();
37                 frame.setVisible(true);
38             } catch (Exception e) {
39                 e.printStackTrace();
40             }
41         });
42     }
43
44     public void reset() {
45         txtNama.setText("");
46         txtEmail.setText("");
47         txtTelepon.setText("");
48         selectedId = null;
49     }
50
51     public void loadTable() {
52         ls = customerRepo.show();
53         TableCustomer tableModel = new TableCustomer(ls);
54         tableCustomer.setModel(tableModel);
55     }
56
57     public CustomerFrame() {
58         setTitle("Manajemen Data Pelanggan");

```

```

59     setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
60     setBounds(100, 100, 556, 586);
61     contentPane = new JPanel();
62     contentPane.setBorder(new EmptyBorder(5, 5, 5, 5));
63     setContentPane(contentPane);
64     contentPane.setLayout(null);
65
66     JLabel lblNama = new JLabel("Nama Pelanggan");
67     lblNama.setFont(new Font("Tahoma", Font.PLAIN, 15));
68     lblNama.setBounds(21, 40, 120, 32);
69     contentPane.add(lblNama);
70
71     JLabel lblEmail = new JLabel("Email");
72     lblEmail.setFont(new Font("Tahoma", Font.PLAIN, 15));
73     lblEmail.setBounds(21, 78, 92, 32);
74     contentPane.add(lblEmail);
75
76     JLabel lblTelepon = new JLabel("No. Telepon");
77     lblTelepon.setFont(new Font("Tahoma", Font.PLAIN, 15));
78     lblTelepon.setBounds(21, 116, 100, 32);
79     contentPane.add(lblTelepon);
80
81     txtNama = new JTextField();
82     txtNama.setBounds(150, 44, 347, 28);
83     contentPane.add(txtNama);
84     txtNama.setColumns(10);
85
86     txtEmail = new JTextField();
87     txtEmail.setColumns(10);
88     txtEmail.setBounds(150, 82, 347, 28);
89     contentPane.add(txtEmail);
90
91     txtTelepon = new JTextField();
92     txtTelepon.setColumns(10);
93     txtTelepon.setBounds(150, 120, 347, 28);
94     contentPane.add(txtTelepon);
95
96     JButton btnSave = new JButton("Save");
97     btnSave.addActionListener(e -> {
98         Customer p = new Customer();
99         p.setNama(txtNama.getText());
100        p.setEmail(txtEmail.getText());
101        p.setTelepon(txtTelepon.getText());
102        customerRepo.save(p);
103        reset();
104        loadTable();
105        JOptionPane.showMessageDialog(null, "Data berhasil disimpan!");
106    });
107     btnSave.setBounds(150, 159, 75, 32);
108     contentPane.add(btnSave);
109
110     JButton btnUpdate = new JButton("Update");
111     btnUpdate.addActionListener(e -> {
112         if (selectedId != null) {
113             Customer p = new Customer();
114             p.setId(selectedId);
115             p.setNama(txtNama.getText());

```

```

116         p.setEmail(txtEmail.getText());
117         p.setTelepon(txtTelepon.getText());
118         customerRepo.update(p);
119         reset();
120         loadTable();
121         JOptionPane.showMessageDialog(null, "Data berhasil diupdate!");
122     } else {
123         JOptionPane.showMessageDialog(null, "Silahkan pilih data yang akan di-update");
124     }
125 }
126 btnUpdate.setBounds(231, 159, 92, 32);
127 contentPane.add(btnUpdate);
128
129 JButton btnDelete = new JButton("Delete");
130 btnDelete.addActionListener(e -> {
131     if (selectedId != null) {
132         int confirmation = JOptionPane.showConfirmDialog(null, "Anda yakin ingin menghapus data ini?", "Konfirmasi Hapus", JOptionPane.YES_NO_OPTION);
133         if (confirmation == JOptionPane.YES_OPTION) {
134             customerRepo.delete(selectedId);
135             reset();
136             loadTable();
137             JOptionPane.showMessageDialog(null, "Data berhasil dihapus!");
138         }
139     } else {
140         JOptionPane.showMessageDialog(null, "Silahkan pilih data yang akan dihapus");
141     }
142 });
143 btnDelete.setBounds(338, 159, 92, 32);
144 contentPane.add(btnDelete);
145
146 JButton btnCancel = new JButton("Cancel");
147 btnCancel.addActionListener(e -> reset());
148 btnCancel.setBounds(429, 159, 92, 32);
149 contentPane.add(btnCancel);
150
151 JScrollPane scrollPane = new JScrollPane();
152 scrollPane.setBounds(21, 222, 580, 318);
153 contentPane.add(scrollPane);
154
155 tableCustomer = new JTable();
156 tableCustomer.addMouseListener(new MouseAdapter() {
157     @Override
158     public void mouseClicked(MouseEvent e) {
159         int row = tableCustomer.getSelectedRow();
160         if (row >= 0) {
161             selectedId = tableCustomer.getValueAt(row, 0).toString();
162             txtName.setText(tableCustomer.getValueAt(row, 1).toString());
163             txtEmail.setText(tableCustomer.getValueAt(row, 2).toString());
164             Object teleponObj = tableCustomer.getValueAt(row, 3);
165             txtTelepon.setText(teleponObj != null ? teleponObj.toString() : "");
166         }
167     }
168 });
169 scrollPane.setViewportView(tableCustomer);
170
171 loadTable();
172 }
173 }

```

Kode ini adalah GUI atau jendela utama untuk aplikasi manajemen pelanggan. Di sini, semua elemen visual seperti label, kolom input (JTextField), tombol, dan tabel (JTable) dibuat dan diatur posisinya. Kode ini juga mengatur logika interaksi, misalnya ketika tombol Save diklik, data dari input akan diambil, dikirim ke CustomerRepo untuk disimpan, lalu tabel diperbarui atau saat sebuah baris tabel diklik, data baris tersebut ditampilkan di kolom input.

Ketika dijalankan, akan menampilkan output seperti ini.

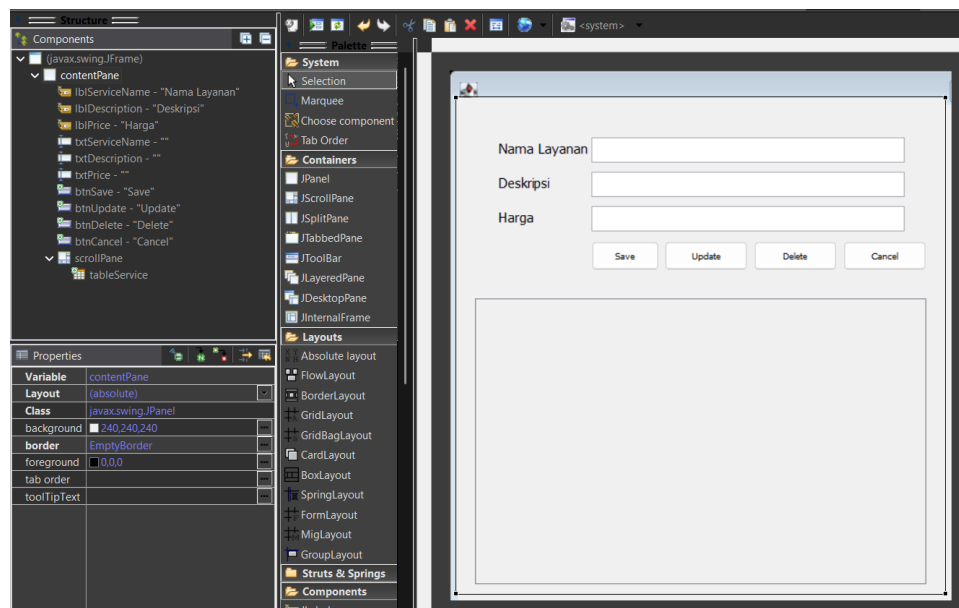
The screenshot shows a Java Swing window titled "Manajemen Data Pelanggan". It contains three text input fields labeled "Nama Pelanggan", "Email", and "No. Telepon". Below these fields are four buttons: "Save", "Update", "Delete", and "Cancel". At the bottom of the window is a table with the following data:

ID	Nama Pelanggan	Email	No. Telepon
6	Iqbal Miftahul Fikri	iqbalmiftahulfikri@gmail.com	081371832190

Setelah di isi, ia akan tersimpan ke database pada phpMyadmin di tabel customer.

	id	nama	email	telepon
☐	Ubah	Salin	Hapus	6 Iqbal Miftahul Fikri iqbalmiftahulfikri@gmail.com 081371832190
☐	Pilih Semua	Dengan pilihan:	Ubah	Salin
			Hapus	Ekspor

9. Buat class JFrame baru pada package ui dengan nama ServiceFrame. Lalu design dan isi kodingan seperti ini.



```

1  package ui;
2
3  import java.awt.EventQueue;
4  import javax.swing.JFrame;
5  import javax.swing.JPanel;
6  import javax.swing.border.EmptyBorder;
7  import DAO.ServiceRepo;
8  import table.TableService;
9  import model.Service;
10 import javax.swing.JLabel;
11 import java.awt.Font;
12 import java.util.List;
13 import javax.swing.JTextField;
14 import javax.swing.JScrollPane;
15 import javax.swing.JButton;
16 import javax.swing.JTable;
17 import java.awt.event.MouseAdapter;
18 import java.awt.event.MouseEvent;
19 import javax.swing.JOptionPane;
20
21 public class ServiceFrame extends JFrame {
22
23     private static final long serialVersionUID = 1L;
24     private JPanel contentPane;
25     private JTextField txtServiceName;
26     private JTextField txtDescription;
27     private JTextField txtPrice;
28     private JTable tableService;
29     private ServiceRepo serviceRepo = new ServiceRepo();
30     private List<Service> ls;
31     private String selectedId;
32
33     public static void main(String[] args) {
34         EventQueue.invokeLater(() -> {
35             try {
36                 ServiceFrame frame = new ServiceFrame();
37                 frame.setVisible(true);
38             } catch (Exception e) {
39                 e.printStackTrace();
40             }
41         });
42     }
43
44     public void reset() {
45         txtServiceName.setText("");
46         txtDescription.setText("");
47         txtPrice.setText("");
48         selectedId = null;
49     }
50
51     public void loadTable() {
52         ls = serviceRepo.show();
53         TableService tu = new TableService(ls);
54         tableService.setModel(tu);
55         tableService.getTableHeader().setVisible(true);
56     }
57
58     public ServiceFrame() {

```

```

59     setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
60     setBounds(100, 100, 556, 586);
61     contentPane = new JPanel();
62     contentPane.setBorder(new EmptyBorder(5, 5, 5, 5));
63     setContentPane(contentPane);
64     contentPane.setLayout(null);
65
66     JLabel lblServiceName = new JLabel("Nama Layanan");
67     lblServiceName.setFont(new Font("Tahoma", Font.PLAIN, 15));
68     lblServiceName.setBounds(47, 40, 100, 32);
69     contentPane.add(lblServiceName);
70
71     JLabel lblDescription = new JLabel("Deskripsi");
72     lblDescription.setFont(new Font("Tahoma", Font.PLAIN, 15));
73     lblDescription.setBounds(47, 78, 92, 32);
74     contentPane.add(lblDescription);
75
76     JLabel lblPrice = new JLabel("Harga");
77     lblPrice.setFont(new Font("Tahoma", Font.PLAIN, 15));
78     lblPrice.setBounds(47, 116, 70, 32);
79     contentPane.add(lblPrice);
80
81     txtServiceName = new JTextField();
82     txtServiceName.setBounds(150, 44, 347, 28);
83     contentPane.add(txtServiceName);
84     txtServiceName.setColumns(10);
85
86     txtDescription = new JTextField();
87     txtDescription.setColumns(10);
88     txtDescription.setBounds(150, 82, 347, 28);
89     contentPane.add(txtDescription);
90
91     txtPrice = new JTextField();
92     txtPrice.setColumns(10);
93     txtPrice.setBounds(150, 120, 347, 28);
94     contentPane.add(txtPrice);
95
96     JButton btnSave = new JButton("Save");
97     btnSave.addActionListener(e -> {
98         Service service = new Service();
99         service.setServiceName(txtServiceName.getText());
100        service.setDescription(txtDescription.getText());
101        try {
102            service.setHarga(Double.parseDouble(txtPrice.getText()));
103            serviceRepo.save(service);
104            reset();
105            loadTable();
106            JOptionPane.showMessageDialog(this, "Data layanan berhasil disimpan!", "Sukses", JOptionPane.INFORMATION_MESSAGE);
107        } catch (NumberFormatException ex) {
108            JOptionPane.showMessageDialog(this, "Harga harus berupa angka.", "Input Error", JOptionPane.ERROR_MESSAGE);
109        }
110    });
111    btnSave.setBounds(150, 159, 75, 32);
112    contentPane.add(btnSave);
113
114    JButton btnUpdate = new JButton("Update");
115    btnUpdate.addActionListener(e -> {
116        if (selectedId != null) {

```

```

117        Service service = new Service();
118        service.setServiceName(txtServiceName.getText());
119        service.setDescription(txtDescription.getText());
120        try {
121            service.setHarga(Double.parseDouble(txtPrice.getText()));
122            service.setId(selectedId);
123            serviceRepo.update(service);
124            reset();
125            loadTable();
126            JOptionPane.showMessageDialog(this, "Data layanan berhasil diupdate!", "Sukses", JOptionPane.INFORMATION_MESSAGE);
127        } catch (NumberFormatException ex) {
128            JOptionPane.showMessageDialog(this, "Harga harus berupa angka.", "Input Error", JOptionPane.ERROR_MESSAGE);
129        }
130    } else {
131        JOptionPane.showMessageDialog(this, "Silahkan pilih data yang akan di-update", "Peringatan", JOptionPane.WARNING_MESSAGE);
132    }
133    });
134    btnUpdate.setBounds(231, 159, 92, 32);
135    contentPane.add(btnUpdate);
136
137    JButton btnDelete = new JButton("Delete");
138    btnDelete.addActionListener(e -> {
139        if (selectedId != null) {
140            int confirmation = JOptionPane.showConfirmDialog(this, "Anda yakin ingin menghapus layanan ini?", "Konfirmasi Hapus", JOptionPane.YES_NO_OPTION);
141            if (confirmation == JOptionPane.YES_OPTION) {
142                serviceRepo.delete(selectedId);
143                reset();
144                loadTable();
145                JOptionPane.showMessageDialog(this, "Data layanan berhasil dihapus!", "Sukses", JOptionPane.INFORMATION_MESSAGE);
146            } else {
147                JOptionPane.showMessageDialog(this, "Silahkan pilih data yang akan dihapus", "Peringatan", JOptionPane.WARNING_MESSAGE);
148            }
149        }
150    });
151    btnDelete.setBounds(330, 159, 92, 32);
152    contentPane.add(btnDelete);
153
154    JButton btnCancel = new JButton("Cancel");
155    btnCancel.addActionListener(e -> reset());
156    btnCancel.setBounds(429, 159, 92, 32);
157    contentPane.add(btnCancel);
158
159    JScrollPane scrollPane = new JScrollPane();
160    scrollPane.setBounds(21, 222, 500, 316);
161    contentPane.add(scrollPane);
162
163    tableService = new JTable();
164    tableService.addMouseListener(new MouseAdapter() {
165        @Override
166        public void mouseClicked(MouseEvent e) {
167            int row = tableService.getSelectedRow();
168            if (row >= 0) {
169                selectedId = tableService.getValueAt(row, 0).toString();
170                txtServiceName.setText(tableService.getValueAt(row, 1).toString());
171                txtDescription.setText(tableService.getValueAt(row, 2).toString());
172                txtPrice.setText(tableService.getValueAt(row, 3).toString());
173            }
174        }
175    });

```

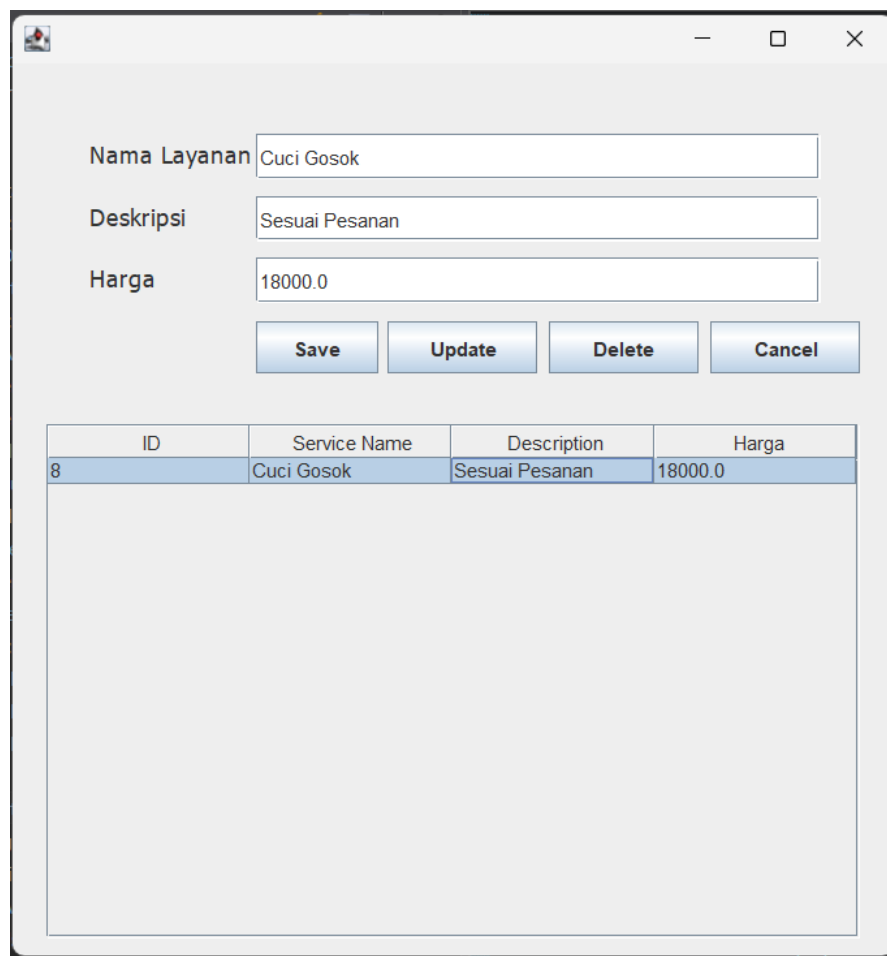
```

175    });
176    scrollPane.setViewportViewView(tableService);
177
178    loadTable();
179    }
180 }

```

Kode ini adalah GUI untuk mengelola data layanan laundry. Di dalamnya, semua elemen visual seperti label, kolom input, tombol, dan tabel sudah disusun dan diatur. Kode ini juga mengatur semua interaksi pengguna, misalnya ketika tombol Save diklik, data dari kolom input akan disimpan ke database melalui ServiceRepo, kemudian muncul notifikasi pop-up dan tabel diperbarui. Selain itu, saat pengguna mengklik baris di tabel, data akan ditampilkan di kolom input untuk bisa diperbarui atau dihapus.

Ketika dijalankan, akan menampilkan output seperti ini.



ID	Service Name	Description	Harga
8	Cuci Gosok	Sesuai Pesanan	18000.0

Setelah di isi, ia akan tersimpan ke database pada phpMyadmin di tabel service.

	id	nama_layanan	deskripsi	harga
☐	8	Cuci Gosok	Sesuai Pesanan	18000.00

☐ Pilih Semua Dengan pilihan: ☐ Ubah ☐ Salin ☐ Hapus ☐ Ekspor

D. Kesimpulan

Dari perancangan dan pembuatan program manajemen data pelanggan menggunakan Java Swing, dapat disimpulkan bahwa aplikasi ini berhasil memberikan kemudahan dalam pengelolaan data secara sederhana namun efisien. Dengan fitur CRUD (Create, Read, Update, Delete), pengguna dapat dengan mudah menambah, melihat, mengedit, dan menghapus data pelanggan melalui antarmuka yang ramah pengguna. Aplikasi ini juga menerapkan prinsip pemrograman berorientasi objek dengan memisahkan tugas sesuai peran masing-masing, yaitu komponen Model yang mengelola data, DAO sebagai penghubung dengan database, dan TableModel untuk menampilkan informasi.

Selain itu, program menyediakan dua metode penyimpanan, baik secara permanen di database maupun sementara di memori, sehingga memudahkan proses pemeliharaan, pengujian, dan pengembangan aplikasi ke depannya. Secara keseluruhan, aplikasi ini tidak hanya menjadi solusi praktis untuk pengelolaan data pelanggan, tetapi juga berfungsi sebagai sarana pembelajaran dalam mengembangkan sistem yang lebih kompleks di masa mendatang.